Teamwork for
Computer and Network Fundamentals
Course 2025-2026

Teamwork Development Report
Phase 1

Made By:
Nigel Xherimeja, UO318641
Julian Rumi, UO318643
Florian Rumi, UO318649
Geri Belba, UO318651

Contents:

1: Correct and Incorrect inputs
2-3: Exercise 1
4: Exercise 2
5: Exercise 3
6: Exercise 4
7: Team report

Correct and incorrect input examples:

ControlWithStrings():

```
Enter first string: YbTqgvPMZ
Enter second string: abcDefghDjklmnp
Access granted
```

```
Enter first string: abcdefg123
Intruder detected
```

```
Enter first string: YbTqgvPMZ
Enter second string: 123
There was some failure
```

BitBasedAccess():

```
Give 2 unsigned integers:
256 256
Access granted
```

```
Give 2 unsigned integers:
256 8
There was some failure
```

```
Give 2 unsigned integers:
8 256
Halt!
```

```
Give 2 unsigned integers:
16 256
Intruder detected
```

AsmZeroBit():

CheckArrayBit():

```
Give 3 8-bit integers:
4 4 13
Access granted
```

```
Give 3 8-bit integers:
3 5 6
These are not the droids you are looking for
```

Exercise 1:

For the local variable of the first string read in the first function of the instructions, indicates its memory address and byte size once the string has been read from keyboard. Include a screenshot that justifies the result.

Memory address: Str1 address is not fixed and depends on the stack during program execution. In the disassembly, the instruction lea eax, [str1] is used to load the address of str1 into the register eax. This means that, at that moment, the register eax contains the memory address of str1. Immediately after, the instruction push eax, passes this address as an argument to the function scanf, allowing the function to store the input string in that memory location. The address of str1 can also be verified by inspecting the value of the eax register after the lea instruction, or by locating the variable in the Memory of the Visual Studio, it is possible to observe the exact memory address where the string is stored once it has been read from the keyboard. We can observe that the 2 results are equal: 001AFEAC

```
    // First string
    printf("Enter first string: ");
00401010  push        404238h
00401015  call        printf (0401AA0h)
0040101A  add         esp,4
    scanf("%19s", str1); // fewer than 20 characters
0040101D  lea         eax,[str1]
00401020  push        eax      ≤ 1ms elapsed
00401021  push        404250h
00401026  call        scanf (0401AF0h)
0040102B  add         esp,8
```

100 %

Registers

```
EAX = 001AFEAC EBX = 00284000 ECX = 001AFE54 EDX = 00000000 ESI = 004024B0
  EDI = 004024B0 EIP = 00401020 ESP = 001AFEAC EBP = 001AFEE0 EFL =
  00000206
```

```
0040100B  call        __CheckForDebuggerJustMyCode (0401BB0h)
    char str1[20];
    char str2[20];

    // First string
    printf("Enter first string: ");
00401010  push        404238h
00401015  call        printf (0401AA0h)
0040101A  add         esp,4
    scanf("%19s", str1); // fewer than 20 characters
0040101D  lea         eax,[str1]
00401020  push        eax      ≤ 1ms elapsed
00401021  push        404250h
00401026  call        scanf (0401AF0h)
0040102B  add         esp,8
```

100 %

Memory 1

Address: 0x001AFEAC Columns: 1

```
0x001AFEAC  05  .
0x001AFEAD  90  ◆
0x001AFEAE  ea  ◆
0x001AFEAF  79  y
```

Byte size: The size of str1 is **20 bytes**, as it is declared as an array of 20 characters(char str1[20]) and each character occupies 1 byte in memory. Even if the string is less than 20 characters, the byte size will not change.

Exercise 3:

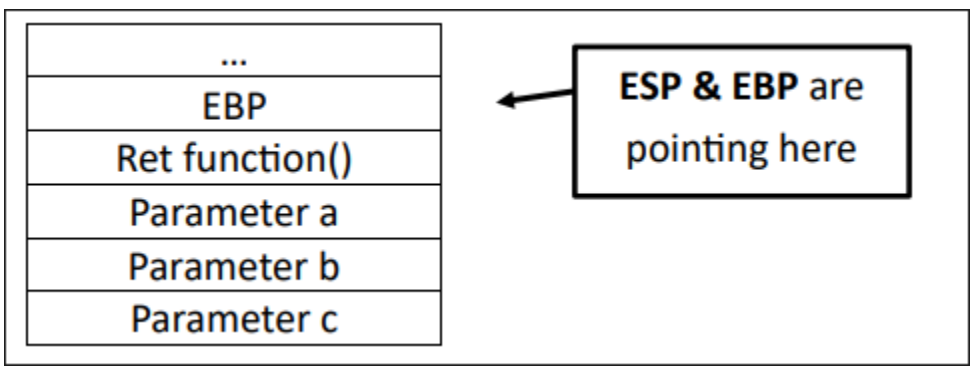
In the IsValidAssembly() function, the parameters are passed in the stack in reverse order (right to left):

```
    IsValidAssembly(3, 1, 6);
0040138D  push        6
0040138F  push        1
00401391  push        3
00401393  call        std::uncaught_exception (0401B48h)
00401398  add         esp,0Ch
```

A bit earlier, EBP (Extended Base Pointer) was pushed in the stack, and then it was given the value of ESP (Extended Stack Pointer):

```
int main() {
00401380  push        ebp
00401381  mov         ebp,esp
00401383  mov         ecx,offset _862E9CC1_Teamwork@cpp (0407079h)
00401388  call        __CheckForDebuggerJustMyCode (0401BC0h)
```

This is the workflow* before and after calling the function:
Push parameters (right to left) -> Call function -> Push EBP -> Set EBP to ESP -> Continue…
The Stack is now ordered in this way:



To access the parameters, we can either increment ESP or get the value of [EBP + n*4], where n is the position on the stack relative to EBP of the parameters. So, parameter a is:

[EBP + 2*4] = [EBP + 8]

EBP is used this way, because typically* it served the purpose of a stable reference point, whilst ESP is more dynamic and is moved from push, pop, inc etc.

*Sources:
https://en.wikibooks.org/wiki/X86_Disassembly/Functions_and_Stack_Frames
https://en.wikipedia.org/wiki/EBP

Exercise 4:

The assembly code of the part where we **read** the value of the vector is as shown below:

```
    if (!(i8arr[0] ^ i8arr[1] ^ i8arr[2] ^ corr)) {
00401309  mov         edx,1
0040130E  imul        eax,edx,0
00401311  movsx       ecx,byte ptr i8arr[eax]
00401316  mov         edx,1
0040131B  shl         edx,0
0040131E  movsx       eax,byte ptr i8arr[edx]
00401323  xor         ecx,eax
00401325  mov         edx,1
0040132A  shl         edx,1
0040132C  movsx       eax,byte ptr i8arr[edx]
00401331  xor         ecx,eax
00401333  movsx       edx,byte ptr [corr]
00401337  xor         ecx,edx
00401339  jne         CheckArrayBit+0B0h (0401350h)
```

Where:

- i8arr -> the 8-bit integer array
- corr -> value equal to ID[1] * 10 + ID[0]

And the registers used:

- edx -> Data register, used for secondary arithmetics
- eax -> Arithmetic register, used for arithmetics
- ecx -> Counter register, used for repeated operations

The program reads the element of the array at index “i” using **movsx**, which moves a value to destination with sign extension. It extends the bitset of the read value based on the sign, since the read value is 1 byte, and the registers store 4 bytes. Ecx is used for the repeated XOR operation.

The compiler sets the value of the index of the array based on its strategy. That is why, for index 0, it multiplies edx by 0 (which it stores in eax, then reads i8arr[eax]), and for index 1 & 2 it shifts left, which, in itself, is multiplication by 2 (2^n if shifting multiple bits), therefore: **edx * 2^0 = 1 & edx * 2^1 = 2**.

Summarizing the flow of reading an element from an array:

1. Set edx to 1
2. Multiply by 0 or shift left (shift left and add 1 for odd indexes)
3. Read element of array at provided index

Sources: https://en.wikibooks.org/wiki/X86_Assembly

Teamwork report:

The teamwork was divided to each member as such:

Nigel Xherimeja:

- Part 1: AsmZeroBit()
- Part 2: Exercise 2
- Time: ~143 min

Julian Rumi:

- Part 1: CheckArrayBit()
- Part 2: Exercise 4
- Teamwork report
- Time: ~62 min

Florian Rumi:

- Part 1: BitBasedAccess()
- Part 2: Exercise 3
- Time: ~59 min

Geri Belba:

- Part 1: ControlWithStrings()
- Part 2: Exercise 1
- Time: ~85 min